

영상처리

5 주차 보고서

컴퓨터융합학부

202102699 정민경

1. 과제 설명

함수이름	my_get_gaussian_filter()
코드설명	이 함수는 gaussian filter를 생성하는 함수이다. 이전 실습과 동일하게 np.mgrid로 x와 y를 구한 다음, gaussian filter를 생성하는 수식으로 gaussian filter를 생성해주었다. 이 때 TODO로 주신 hint에서처럼 정규화는 진행하지 않고 filter를 return 해주었다.
이미지	<pre> y, x = np.mgrid[-(f_h // 2):(f_h // 2) + 1, -(f_w // 2):(f_w // 2) + 1] # 2차 gaussian mask 생성 gaussian_filter = np.exp(-((x * x) + (y * y))) / (2 * sigma * sigma) # 정규화 하지 않음 # # mask 총합 1 : 평균 밝기의 변화가 없도록 하기 위함 # gaussian_filter = gaussian_filter / np.sum(gaussian_filter) return gaussian_filter </pre>

함수이름	first_DoG_filter_mask()
코드설명	미분필터를 사용해 gaussian_filter 를 filtering 한 DoG filter를 구하는 함수이다. 이 함수에서 나는 미분필터를 3 * 3 filter를 사용하였고, fsize * fsize 크기의 gaussian_filter를 사용하면 영역을 넘어가는 부분이 생기기 때문에 위, 아래, 양옆을 각각 1 pixel 씩 더 큰 gaussian_filter를 사용하였다. (fsize + 2 크기의 gaussian_filter 사용) gaussian_filter 와 derivative_filter를 x, y 각각 생성했다면, gaussian_filter 와 derivative_filter를 filtering 시킨 결과 filter (= gaussianfilter_x, gaussianfilter_y)를 return 해주었다.
이미지	<pre> gaussian_filter = my_get_Gaussian_filter((fsize+2, fsize+2), sigma) derivative_filter_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]) derivative_filter_y = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]]) (x_h, x_w) = derivative_filter_x.shape_# (3, 3) (y_h, y_w) = derivative_filter_y.shape_# (3, 3) gaussianfilter_x = np.zeros((fsize, fsize)) gaussianfilter_y = np.zeros((fsize, fsize)) for row in range(fsize): for col in range(fsize): gx_filter = gaussian_filter[row: row + x_h, col: col + x_w] gaussianfilter_x[row, col] = np.sum(np.multiply(gx_filter, derivative_filter_x)) gy_filter = gaussian_filter[row: row + y_h, col: col + y_w] gaussianfilter_y[row, col] = np.sum(np.multiply(gy_filter, derivative_filter_y)) return gaussianfilter_x, gaussianfilter_y </pre>

함수이름	second_DoG_filter_mask()
코드설명	이 함수는 gaussian filter 를 수식을 사용해 미분해 DoG filter 를 return 하는 함수이다. 이론자료 및 실습자료에 정의되어있는 gaussian filter의 1차 미분수식을 사용해 x, y 각각 값을 구한 뒤 return 해주었다.

이미지

```
y, x = np.mgrid[-(fsize // 2):(fsize // 2) + 1, -(fsize // 2):(fsize // 2) + 1]

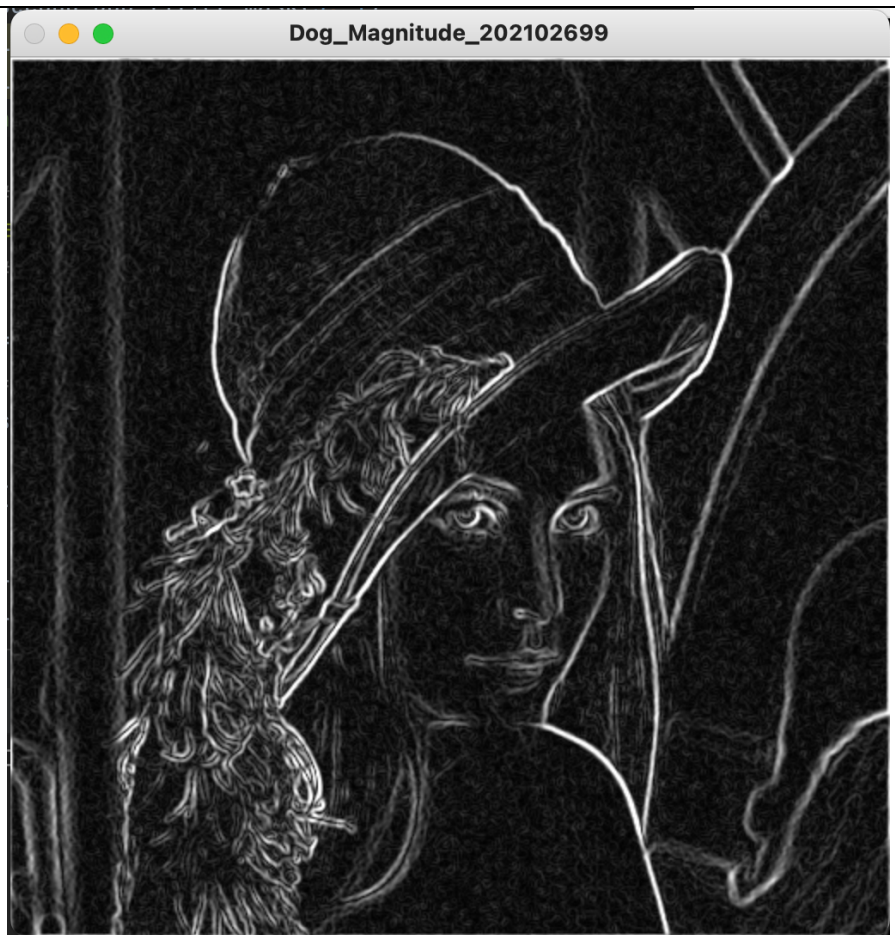
# 이게 정답 코드
DoG_x = (-x / (sigma ** 2)) * (np.exp(-((x ** 2) + (y ** 2)) / (2 * sigma ** 2)))
DoG_y = (-y / (sigma ** 2)) * (np.exp(-((x ** 2) + (y ** 2)) / (2 * sigma ** 2)))

return DoG_x, DoG_y
```

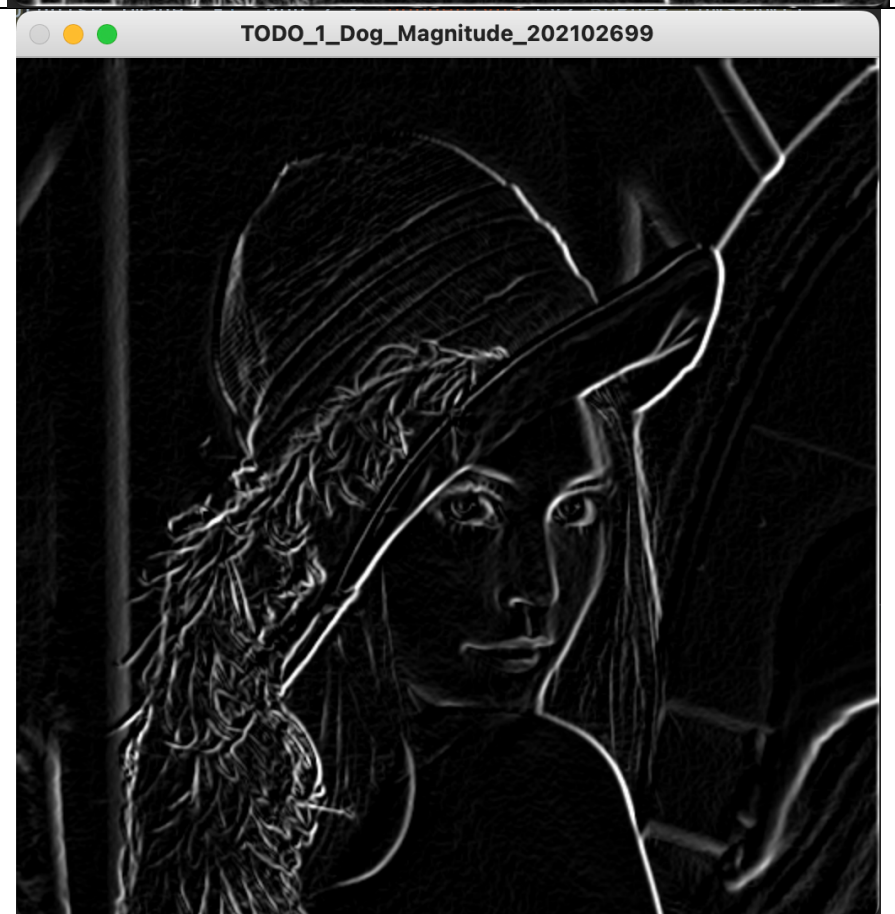
2. 결과 이미지



DoG_magnitude



Dog_magnitude
(Dog filter)



Dog_magnitude
(Dog expression)



TODO_2_Dog_Magnitude_202102699



3. 이미지 분석

- **Sobel filter 와 DoG filter**

- Noise 가 생긴 이미지를 sobel filter 와 DoG filter를 각각 적용시켜 이미지를 생성했을 때 DoG filter 가 sobel filter 보다 noise가 더 적은 것을 확인할 수 있다. 또한 noise가 더 적기 때문에 edge 가 더 선명하게 보인다.

- **미분필터를 적용한 DoG_filter와 gaussian을 수식으로 미분한 DoG_filter**

- TODO1 image는 gaussian filter 를 직접 수식으로 미분해 구한 DoG_filter 를 사용한 image이고, TODO2 image는 gaussian filter 에 미분필터로 filtering 해 구한 DoG_filter 를 사용한 image이다. TODO1 image보다 TODO2 image가 edge 가 선명하게 생성된 것을 결과 이미지를 통해 확인할 수 있다.